Introdução à Orientação por Objetos Classes & objetos

Aula 6

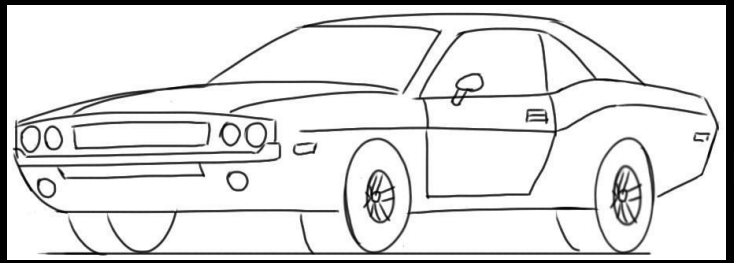
Universidade de Brasilia
Faculdade do Gama

Orientação a Objetos

Prof. André Lanna

Da aula passada...

Instanciação de classes = criação de objetos



Classe Veículo

Objetos do tipo Veículo
(instâncias de Veículo)

Métodos construtores

Métodos construtores são métodos utilizados para a **instanciação** de objetos a partir de uma classe.

Em Java são métodos com o mesmo nome da Classe à qual pertence e acompanhados do operador **new**.

Tipos de métodos construtor:

- Padrão
- Alternativo

Construtores

Veículo
marca : **String** modelo: **String** cor: **String** capacidadeCarga: **int** velocidade: **float**
Veiculo() **Veiculo(String mrc, String mdl, String cor)** acelerar(acel: float) : **float** frear() : **float** ligarDesligar(): **boolean**

Construtor **padrão**:

- Não há parâmetros em sua assinatura;
- Inicializa atributos com valores padrão
- Sua declaração e implementação não é obrigatória.

Construtor **alternativo**:

- Há parâmetros em sua assinatura;
- Geralmente utilizado para inicializar atributos com valores passados por parâmetros;
- Sua declaração e implementação é opcional;
- Uma classe pode ter quantos construtores alternativos forem necessários;

Construtores

Veículo
marca : **String** modelo: **String** cor: **String** capacidadeCarga: **int** velocidade: **float**
Veiculo() **Veiculo(String mrc, String mdl, String cor)** acelerar(acel: float) : **float** frear() : **float** ligarDesligar(): **boolean**

```
Class Veiculo {

    //construtor padrão
    Veiculo() {
        //implementação opcional
    }


    //Construtor alternativo
    Veiculo(String mrc,
            String mdl,
            String c) {

        marca = mrc;
        modelo = mdl;
        cor = c;
    }

    ...
}
```

Construtores - Instanciação de objetos.

Veículo
marca : **String** modelo: **String** cor: **String** capacidadeCarga: **int** velocidade: **float**
Veiculo() **Veiculo(String mrc, String mdl, String cor)** acelerar(acel: float) : **float** frear() : **float** ligarDesligar(): **boolean**

```
public static void main() {
    Veiculo v1, v2;

    v1 = new Veiculo();
    v2 = new Veiculo("Fiat",
"Palio", "Prata");

    system.out.println(v1.marca);
    system.out.println(v1.modelo);
    system.out.println(v1.cor);

    system.out.println(v2.marca);
    system.out.println(v2.modelo);
    system.out.println(v2.cor);

}
```

Referências / associações entre objetos

Relacionamentos entre objetos

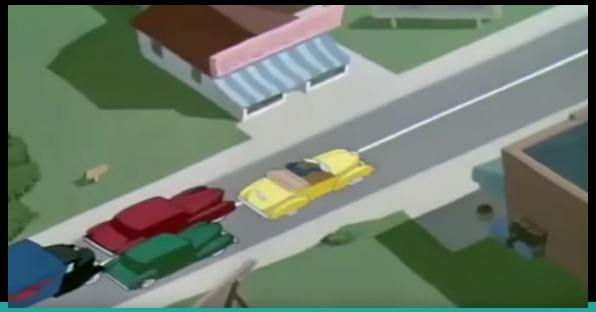
- Objetos não existem “isolados” de demais objetos! Olhe à sua volta:
- Eles se relacionam para formar estruturas maiores.

- Mesa “possui” 4 pernas
- Mesa “possui” 4 cadeiras
- Mesa “possui” tampo

- Sala de aula “possui” 1 quadro
- Sala de aula “possui” 40 carteiras
- Cada carteira “possui” 1 cadeira
- Sala de aula “possui” 1 porta.

Relacionamentos entre objetos

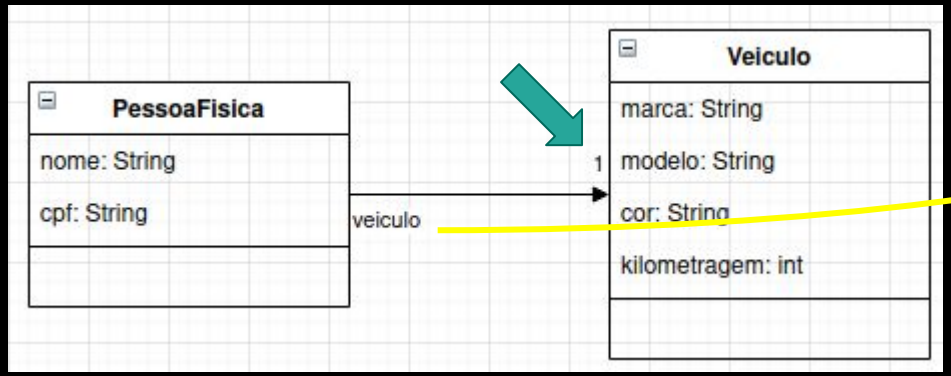
- Objetos não existem “isolados” de demais objetos! Olhe à sua volta:
- Objetos se relacionam para executarem tarefas que isoladamente não seria possível.



Relações entre objetos

Como representar a relação entre objetos através de um diagrama de classes?
- Através de referência(s) para a classe de destino.

Associação **unidirecional**:



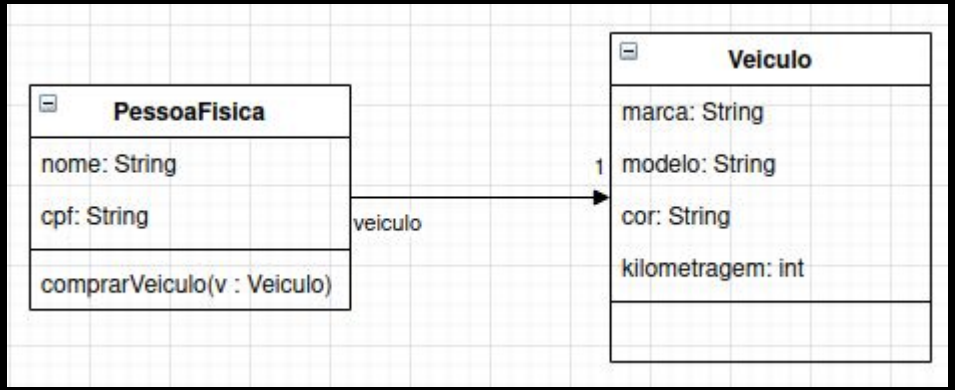

```
class PessoaFisica {
    String nome;
    String cpf;

    Veiculo veiculo;
}
```

A referência `veiculo` é um atributo da classe PessoaFisica!

Relações entre objetos

Como associar um objeto Veículo a um objeto PessoaFisica?

- via atributo ou
- via método (preferível)

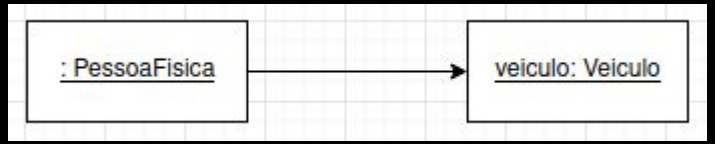



```
class PessoaFisica {
    String nome;
    String cpf;
    Veiculo veiculo;

    void comprarVeiculo(Veiculo v) {
        veiculo = v;
    }
}
```

Método `comprarVeiculo()` é o responsável por associar um objeto `Veiculo` a um objeto `PessoaFisica`.

Relações entre objetos

Em termos de objetos a relação pode ser representada via Diagrama de Objetos:



Como entender esse diagrama?

Um objeto de `PessoaFisica` está relacionado à um objeto `Veículo` através da referência `veiculo`.

Questões:

a) O que significa a linha sublinhada embaixo do nome da classe nesse diagrama?

b) O que significa o identificador antes dos : <*Nome da classe*>? (veiculo:Veiculo, no caso desse diagrama)

c) O que significa não ter um identificador antes dos : como no caso de PessoaFisica?

Comunicação entre objetos

Comunicação entre objetos

Durante a execução de um programa diversos objetos são instanciados, cada um com seus atributos e métodos.

A execução do programa se dá pela execução dos métodos de cada um dos objetos e da comunicação existente entre eles.

A comunicação entre objetos se dá através de **troca de mensagens**.

Formato de uma mensagem:

```
objeto.métodoChamado(par1, …, parN);
```

destinatário

método a ser chamado

lista de parâmetros

Comunicação entre objetos

Diagrama de Classes:

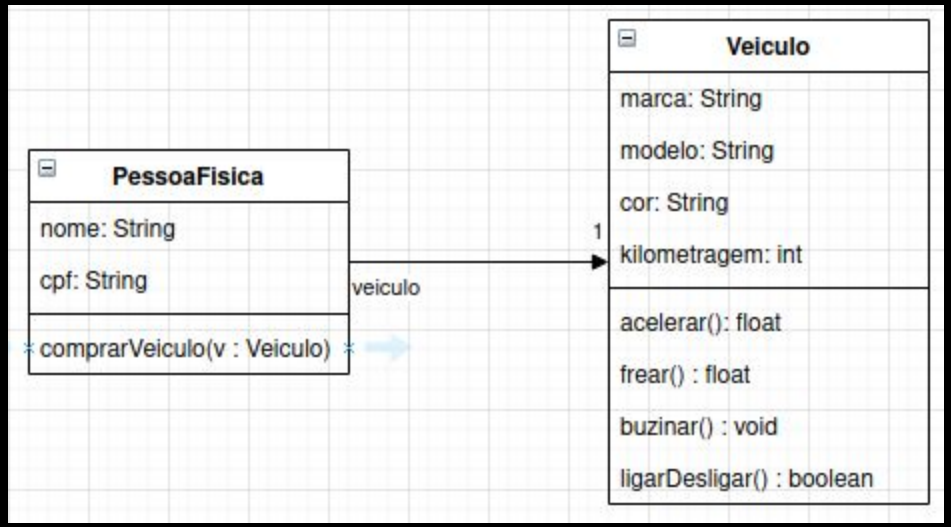
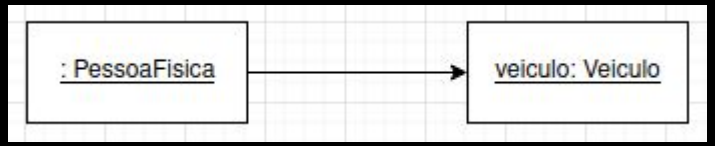


Diagrama de Objetos:



O objeto `:PessoaFisica` pode enviar mensagens ao objeto `:Veiculo` através da referência `veiculo`.

Exemplo:

```
veiculo.acelerar();
veiculo.frear();
veiculo.buzinar();
veiculo.ligarDesligar();

veiculo.kilometragem = 0;
```

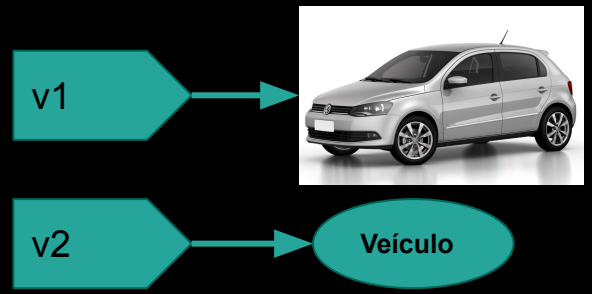
Estado de um objeto

Objetos possuem **estado**: conjunto de valores de seus atributos em um instante de tempo.

- Mudou o valor de um atributo, mudou o estado do objeto.

Exemplo:

```
//instanciação do veículo 2
Veiculo v2 = new Veiculo();
```




```
//Definindo as características de v2
v2.setMarca("Hyundai");
v2.setModelo("HB20");
v2.setCor("Cinza");
v2.setCapacidadeCarga(380);
v2.setVelocidade(0);
```

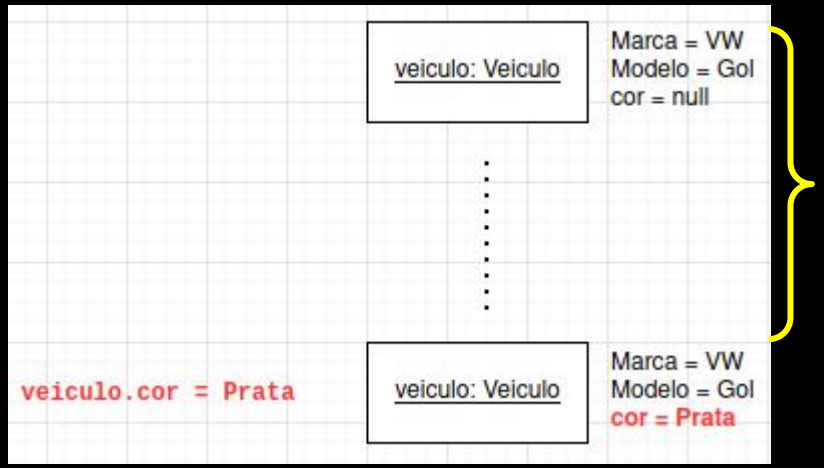
O que aconteceu com o estado de v2 durante a execução dessas instruções?

E o que aconteceu com o estado de v1 durante essas instruções?

Retenção de estado

Capacidade de um objeto manter o seu estado inalterado durante o tempo em que estiver em memória e sem receber um estímulo externo (mensagem).

O estado de um objeto só será alterado no momento em que houver uma instrução explícita que o faça mudar o valor de algum atributo.



Retenção de estado!
O objeto não alterou seu estado até o momento em que uma mensagem foi enviada a ele.

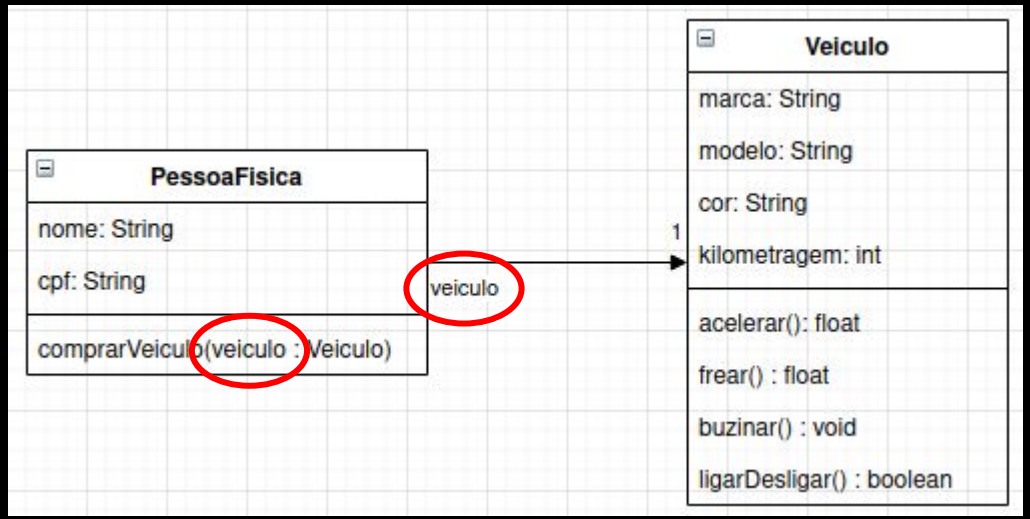
(?) Métodos **`getAtributo()`** são capazes de alterar o estado de um objeto?

Referência *this*

Referência *this*

Recurso presente na linguagem Java. Recursos semelhantes podem ser encontrados em outras linguagens OO.

Cada objeto possui uma referência dele próprio.
Essa referência é acessada através da palavra-reservada *this*.

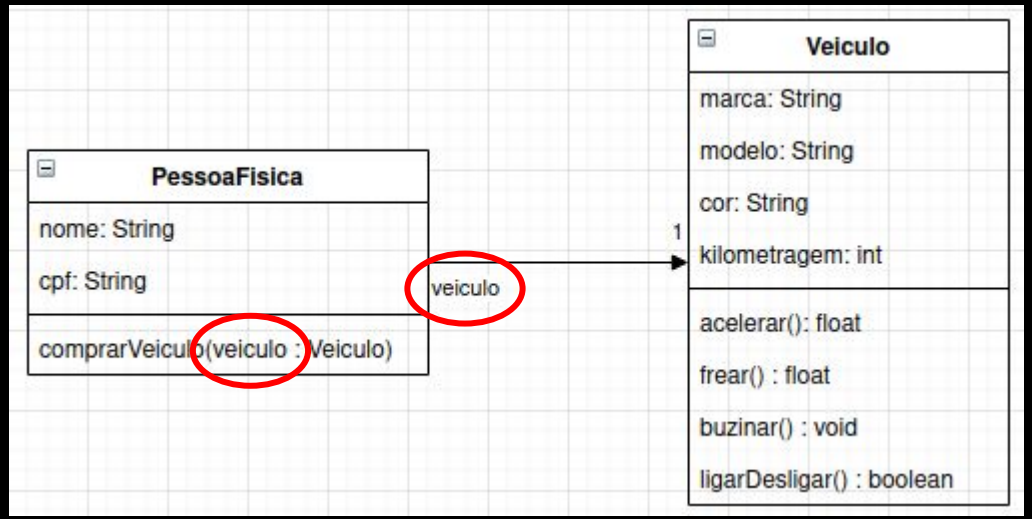


No escopo do método `comprarVeiculo()` há dois identificadores `veiculo`:

- O atributo `veiculo` de PessoaFisica
- O parametro `veiculo` de comprarVeiculo()

Como distingui-los?

Referência *this*

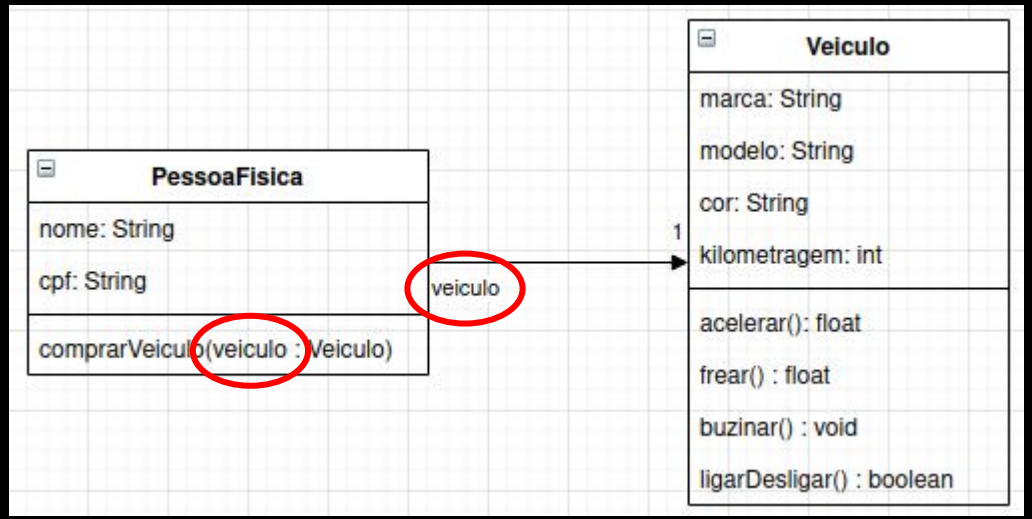



```
class PessoaFisica {
    String nome, cpf;
    Veiculo veiculo;
    ...
    comprarVeiculo (Veiculo veiculo) {
        veiculo = veiculo;
    }
    ...
}
```

Essa distinção de referências não é possível de ser realizada pelo computador.

Como resolver?
Via referência *this*.

Referência *this*




```
class PessoaFisica {
    String nome, cpf;
    Veiculo veiculo;
    ...
    comprarVeiculo (Veiculo veiculo) {
        this.veiculo = veiculo;
    }
    ...
}
```

`this.veiculo` refere-se à referência `veiculo` do objeto.

`veiculo` é a referência definida como parâmetro do método `comprarVeiculo`.

Destrutores
Coletor de lixo

Destrutores

Objetos ficam "inertes" na memória esperando alguma mensagem.

- Para sempre? Até quando? E se ele não precisar mais ser utilizado?

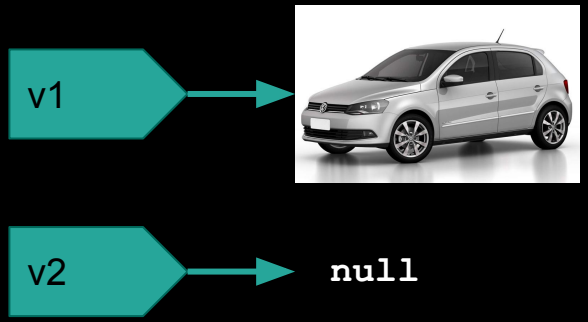
Métodos destrutores implementam um comportamento a ser executado quando um objeto estiver prestes a ser destruído. Ex.:

- Fechar arquivo, fechar conexão com banco, etc...

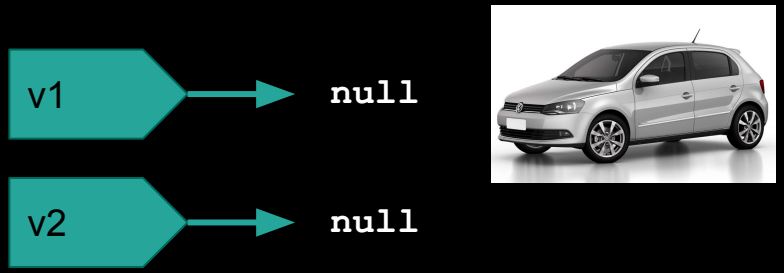
Algumas linguagens OO oferecem métodos para destruir objetos à força. Ex.: C++

- Java não destrói objetos à força, mas especifica seu comportamento quando for destruído → Método **finalize**.
- Quem destrói objetos em Java é o coletor de lixos (*Garbagge collector*) de modo automático.

Acesso adequado a membros de um objeto




```
//perda da referencia de v1
v1 = null;
```

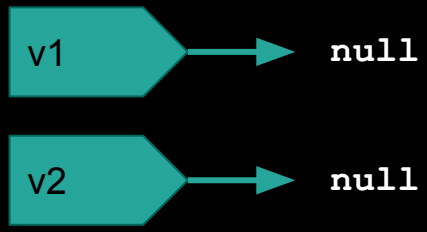


Esse veículo não é referenciado por ninguém nesse momento.

Coletor de lixo o identifica e elimina.

```
//chamada explicita ao GC
System.gc();
```

GC chama `finalize`, destrói objeto e libera espaço em memória.



Exercícios